Taller Docker Informe y justificación

Eco Air Backend | Estudiante: Nicolas Federico Delgado Perez | Código: 841024 | 20 de octubre de 2026

Resumen del resultado

Se construyó una imagen de dos etapas con Maven 3.9.6 y Java 17. El runtime contiene el JRE y el JAR ejecutable, ejecuta Java con UID 10001 y recibe OPERATOR_ID=841024. La API respondió desde el contenedor. En la prueba de 48 MiB, Docker informó ExitCode 137 y OOMKilled: true. El menor límite que arrancó en menos de 15 segundos y siguió activo tras 45 segundos, confirmado en dos ejecuciones, fue 89 MiB en este equipo.

Origen del proyecto y diseño de la imagen

El repositorio inicial solo tenía un README. Se añadió una aplicación Spring Boot 3.3.4 con Hibernate, H2 en memoria y una API mínima de alertas para poder construir y auditar la imagen. En el Dockerfile, el archivo pom.xml se copia antes de src/ y se ejecuta mvn dependency:go-offline antes de compilar; por ello la descarga queda en caché cuando cambia el código. La etapa final usa eclipse-temurin:17-jre-alpine y copia únicamente el JAR. La propiedad del directorio /app y el proceso pertenecen a sensor_user.

Identidad y control de privilegios

Comprobación	Evidencia obtenida
Hash de imagen	sha256:c0d483fd40797eb21dabf25c22ad32bb2e6f0ce509fb2995b35c2cf4042e6fb2
Proceso Java	docker top: USER 10001; id: sensor_user / sensorgroup
Escritura en /etc	Permission denied; código de salida 1
Herramientas y fuentes	which mvn sin salida; find no devolvió archivos .java
Operador	OPERATOR_ID=841024

La denegación de escritura demuestra que el proceso no puede modificar ese directorio del sistema. La ausencia de Maven concuerda con la separación de etapas. El comando find terminó con código 1 porque el usuario carece de acceso a algunos directorios; el Dockerfile confirma además que no copia src/ al runtime.

Memoria y arranque

Límite	Estado	Inicio	Memoria al medir
48 MiB	OOMKilled	Sin inicio	ExitCode 137
88 MiB	OOMKilled	Sin inicio	0B tras salir
89 MiB	Activo a 45 s	13.364 s / 12.823 s	83.05 / 89 MiB en repetición
100 MiB	Activo a 45 s	9.349 s	88.12 / 100 MiB
256 MiB	Activo	7.046 s	157.9 / 256 MiB
384 MiB	Activo	4.756 s	167.4 / 384 MiB
512 MiB	Activo	5.134 s	175.3 / 512 MiB

El resultado de 48 MiB no fue instantáneo: la inspección a los 10 segundos aún mostró running; la consulta posterior mostró OOMKilled: true. El límite de 87 MiB cumplió en un intento y tardó 16.341 segundos en otro. Por eso se fijó 89 MiB como el mínimo observado y repetido con resolución de 1 MiB. En producción conviene dejar margen: 100 MiB consumió menos proporción del cgroup y arrancó más rápido.

Salidas decisivas de Docker

```
docker inspect eco_air_crash
Status: exited | ExitCode: 137 | OOMKilled: true
docker top eco_air_active -o pid,user,comm
PID USER COMMAND
2996 10001 java
docker exec eco_air_active touch /etc/crontab_fake
touch: cannot touch '/etc/crontab_fake': Permission denied
```

Comprobación final

El contenedor final eco_air_active quedó activo con -m 89m y puerto 8080. Docker registró Started EcoAirApplication in 14.234 seconds. Las columnas principales de docker stats --no-stream fueron:

```
CONTAINER ID NAME CPU % MEM USAGE / LIMIT MEM %
bbba1e768bad eco_air_active 0.16% 82.02MiB / 89MiB 92.16%
```

La API devolvió [] en GET /alerts. Los comandos, timestamps, IDs de contenedor, hashes de capas y salidas íntegras están en FORENSIC_REPORT.md y audit_session.log. El umbral medido depende del motor Docker, la carga y la configuración de la JVM; no debe tomarse como una garantía universal.